DESARROLLO DE APLICACIONES II

Catálogos globales: barrios, zonas y listas de referencia

Módulo 9 · Core

20 de octubre de 2026

Contenido

Por qué existe esto 3
Barrios 3
Zonas 4
Catálogos globales 4
Quién puede escribir 4
Por qué REST y no eventos 5
Cómo lo probás hoy 5
Lo que falta 5

Para los 9 equipos. El Core ya expone el catálogo territorial y las listas de referencia compartidas. Esto reemplaza cualquier lista de barrios que tengan escrita a mano.

> **Si sólo te interesa una cosa:** `GET /api/v1/catalogos/barrios` con el token de tu módulo. Devuelve los 16 barrios con un identificador estable. Guardá el `codigo` (o el `id`) en tus datos, no el nombre.

Por qué existe esto

El enunciado se lo asigna al Core: *«Barrios, zonas y catálogos globales»* (sección 10, módulo 9). Es el único dato de negocio que el Core administra.

La razón práctica: si cada módulo mantiene su propia lista de barrios, dos eventos que hablan del mismo lugar llegan con identificadores distintos y no se pueden cruzar. Atención Ciudadana necesita mostrar un tablero por barrio, Obras ubica sus órdenes de trabajo, Ambiente arma recorridos agrupando barrios: los tres tienen que estar hablando de los mismos barrios.

Nos lo venían pidiendo desde agosto y habíamos contestado mal. Dijimos que no era nuestro. Lo era. Perdón por la vuelta.

Barrios

```
GET /api/v1/catalogos/barrios
Authorization: Bearer <token de tu módulo>
```


```
[
  {
    "id": "f9ce3ea7-8806-4ba6-ae6e-f2446eae8126",
    "codigo": "alto-verde",
    "nombre": "Alto Verde",
    "zona": "norte",
    "zonaNombre": "Zona Norte",
    "metadatos": {},
    "activo": true
  }
]
```


Filtro	Para qué
`?zona=ribera`	Sólo los barrios de una zona
`?q=villa`	Buscar por nombre, sin distinguir mayúsculas
`?soloActivos=true`	Sólo los vigentes

Y uno solo: `GET /api/v1/catalogos/barrios/{codigo}`.

Los dos identificadores son estables

`id` (UUID) y `codigo` (slug legible) sirven los dos, y **ninguno se reusa nunca**. Guardá el que te sea más cómodo; nosotros usamos el `codigo` en los ejemplos porque se lee.

Lo que no hay que guardar es el `nombre`. Un barrio se puede renombrar; su identificador no cambia.

Nada se borra

Un barrio que deja de existir queda con `activo: false`, pero **se sigue pudiendo consultar**. Eso es a propósito: hay eventos históricos que lo referencian y tenés que poder resolverlos.

Si listás con `?soloActivos=true` no aparece, pero `GET /catalogos/barrios/{codigo}` lo sigue devolviendo. Y su código no se puede reusar para un barrio nuevo.

Cachearlo está bien

Es una lista chica y cambia muy poco. Leela al arrancar tu módulo y refrescala cada tanto. No hace falta pegarle en cada request.

Zonas

```
GET /api/v1/catalogos/zonas
```

```
[ { "codigo": "ribera", "nombre": "Ribera", "barrios": 3, "activo": true } ]
```

Hoy son 5: `centro`, `norte`, `sur`, `oeste`, `ribera`.

Ojo con esto: son zonas *administrativas*

Las zonas del Core agrupan barrios y nada más. Las zonas *operativas* de cada área —los recorridos y frecuencias de recolección de Ambiente, por ejemplo— son de ese módulo, se arman agrupando estos barrios, y el Core no las administra.

Si tu módulo tiene una noción propia de zona, convive con esta: usá nuestros barrios como unidad mínima y agrupalos como necesites.

Catálogos globales

Las listas de referencia que más de un módulo necesita escribir igual.

```
GET /api/v1/catalogos                      → qué catálogos existen
GET /api/v1/catalogos/{catalogo}/items     → el contenido de uno
```

Hoy hay cuatro:

Catálogo	Items
`categorias-reclamo`	alumbrado, recoleccion, calzada, arbolado, agua, transito, espacios-verdes, otros
`prioridades`	baja, media, alta, urgente
`tipos-documento`	dni, cuit, cuil, pasaporte
`estados-tramite`	iniciado, en-proceso, derivado, resuelto, rechazado

Cada item trae `codigo`, `nombre`, `descripcion`, `orden` y `metadatos`.

`orden` existe para que los nueve módulos muestren la lista igual. Vienen ordenados; respetá ese orden en tus combos y no los alfabetices.

El mismo `codigo` puede existir en dos catálogos distintos: la unicidad es por catálogo, no global.

Quién puede escribir

Leer: cualquier módulo autenticado. Escribir: sólo el Core.

Un catálogo compartido deja de servir en el momento en que cada equipo puede cambiarlo: lo que uno corrige, otro lo pisa. Si intentás un `POST` o un `PATCH` con el token de tu módulo recibís un **403**.

Cómo pedir un cambio

Escribinos con: qué falta o qué está mal, y el `codigo` que proponés si es un alta. Lo cargamos y queda disponible para todos en el acto.

Los códigos son slugs: minúsculas, números y guiones simples (`villa-elisa` , `espacios-verdes`). **Se validan, no se auto-corrigen**: si mandás `Villa Elisa` o `Villa-Elisa` recibís un 422 en vez de que lo guardemos distinto de como lo mandaste.

Por qué REST y no eventos

Un catálogo es configuración, no un hecho de negocio. El enunciado habilita la integración sincrónica justamente para esto:

> *«Se permitirá mediante APIs REST únicamente cuando la respuesta sea necesaria para continuar inmediatamente con una operación, por ejemplo para validar una identidad, **consultar una configuración** o verificar una condición indispensable.»* (sección 7)

Así que **el Core sigue sin publicar eventos de negocio**. No esperen un `barrioCreado` por la cola.

Cómo lo probás hoy

No hace falta esperar al deploy. El Core levanta local:

```
cd backend
.venv/bin/python -m app.seeds
.venv/bin/uvicorn app.main:app --reload
```

Swagger en `http://localhost:8000/docs` , sección **Catálogos globales**. Tu usuario es `<modulo>@muni.uade.edu.ar` / `Cambiala123` .

```
TOKEN=$(curl -s -X POST localhost:8000/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"ambiente@muni.uade.edu.ar","password":"Cambiala123"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["accessToken"])')

curl -s localhost:8000/api/v1/catalogos/barrios -H "Authorization: Bearer $TOKEN"
```

Lo que falta

Los datos de arranque son ficticios. Las 5 zonas y los 16 barrios son una base coherente para que puedan integrar hoy, no el padrón real. Cuando el equipo o la cátedra definan la lista de verdad, la cargamos por la API sin tocar código — y los `codigo` que ya hayan guardado siguen siendo válidos si los nombres coinciden.

Si tu módulo necesita un catálogo que no está, decinos cuál. Agregar uno es cargar su primer item; no hay migración de por medio.